Anchorpoint

Software Requirements Specification

V1

2/28/26

Haiden Hunter
Scrum Master / Product Owner

Daley Fraser
Lead Software Engineer

Dante Samarco
Software Engineer / UX & Engagement Engineer

Gavin Fehl
Test Engineer / Software Engineer

Prepared for
Animal Intelligence / SWE CS 1530 Spring 2026

Revision History

Date	Description	Author	Comments
2/28/26	Version 1	All of Group 9	Initial Draft

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date

Table of Contents

Document Approval 2
1. Introduction 1
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
2. General Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 Constraints, Assumptions and Dependencies 3
3. Specific Requirements 3
3.1 External Interface Requirements 3
3.1.1 Software Interfaces 3
3.2 Functional Requirements 3
3.2.1 Chat Interface 3
3.2.2 Private Personalization Input 3
3.2.3 Daily Report Creation 3
3.2.4 Deep Research 3
3.2.5 Voice Output 4
3.2.6 Graceful Tool Failure 4
3.2.7 Contextual Memory 4
3.2.8 Model Selection and Management 4
3.2.9 Web Search 4
3.2.10 Conversation History 4
3.3 Use Cases 4
3.3.1 Use Case #1 4
3.3.2 Use Case #2 5
3.4 Classes / Objects 6
3.4.1 ChatSession 6
3.4.1.1 Attributes 6
3.4.1.2 Methods 6
3.4.2 Message 6
3.4.2.1 Attributes 7
3.4.2.2 Methods 7
3.4.3 UserProfile 7
3.4.3.1 Attributes 7
3.4.3.2 Methods 7
3.4.4 MemoryManager 7
3.4.4.1 Attributes 7
3.4.4.2 Methods 7

3.4.5 MemoryEntry.....8
3.4.5.1 Attributes.....8
3.4.5.2 Methods.....8
3.4.6 LLMInterface.....8
3.4.6.1 Attributes.....8
3.4.6.2 Methods.....8
3.4.7 BriefingService.....8
3.4.7.1 Attributes.....9
3.4.7.2 Methods.....9
3.4.8 TTSService.....9
3.4.8.1 Attributes.....9
3.4.8.2 Methods.....9
3.5 Non-Functional Requirements.....9
3.5.1 Performance.....9
3.5.2 Reliability.....9
3.5.3 Availability.....10
3.5.4 Security.....10
3.5.5 Maintainability.....10
3.5.6 Portability.....10
3.6 Logical Database Requirements.....10
3.7 Other Requirements.....11
4. Analysis Models.....11
4.1 Use Case Diagrams.....12
4.1.1 Use Case #1 Diagram.....12
4.1.2 Use Case #2 Diagram.....12
4.2 Class Diagrams.....13
4.3 Sequence Diagrams.....13
4.4 Activity Diagrams.....13
4.5 State-Transition Diagrams (STD).....13
5. Change Management Process.....13
A. Appendices.....13

1. Introduction

Anchorpoint is a privacy first personal AI assistant that runs entirely on the user's local machine. It leverages local large language models to deliver features traditionally believed to only belong to cloud based AI services, including deep research, contextual memory, advanced morning briefings, and custom voice interactivity, without ever sending one's personal data to external servers.

1.1 Purpose

This Software Requirements Specification document describes the functional and non-functional requirements, as well as providing additional visuals and diagrams, for Anchorpoint. It is intended for our development team here at Animal Intelligence and any stakeholders evaluating the system. This document serves as the authoritative reference for all design, implementation, and testing decisions throughout the project's lifecycle.

1.2 Scope

Anchorpoint is a browser based AI assistant designed to be highly customizable, extremely capable, and completely private, powered exclusively by local LLMs.

Anchorpoint will provide a chat interface for conversation with a local LLM, support deep research with web search and source citations, offer customizable voice output to support verbal back and forth communication, guide the input of specific personal information in order to build specialized day to day reports by aggregating calendar, weather, and news information, and maintain contextual memory across chats to fully utilize that aforementioned personal information. Anchorpoint will not send any PII out to the internet, offer cloud or remote access, or assist with tasks that might be dangerous or require professional guidance.

Our ultimate goal is to push what local LLMs can achieve and make Anchorpoint comparable in common use cases to models developed by companies like ChatGPT, Anthropic, and Google.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
AI	Artificial Intelligence
LLM	Large Language Model
PII	Personally Identifiable Information
TTFT	Time to First Token
API	Application Programming Interface
TTS	Text to Speech
Ollama	Open source tool for downloading, managing, and running LLMs locally.

SRS	Software Requirements Specification / This Document
Inferencing	The process of using a trained LLM to generate text or other outputs based on new, user provided inputs. This is the term used to describe the actual use of an LLM.
Animal Intelligence	Name of Group 9, the creators of this software and composers of this document.
SSE	Server Send Events
RAM	Random Access Memory

2. General Description

This section of the SRS does not state specific requirements, rather it describes the general factors that might affect Anchorpoint and its requirements in order to make the actual requirements, found in subsection 3, easier to understand.

2.1 Product Perspective

Anchorpoint exists in a market dominated by cloud based AI assistants such as ChatGPT, Claude, and Google Gemini. These products require users to send their personal data, such as calendar events, financial goals, and project details, to remote servers that store their chat history in order to fully utilize the benefits of a personalized AI assistant. Anchorpoint offers a fundamentally different approach. All inference, personalization, and data storage happens on the user's device. The system builds on existing open source infrastructure, using Ollama as the local LLM inference backend, and integrating external APIs only for data retrieval, never for processing user context.

2.2 Product Functions

Fundamentally, Anchorpoint is a conversational AI chat interface connected to a locally running LLM. Anchorpoint offers an automated daily report that combines calendar events, weather forecasts, news headlines, and commute data into a single cohesive briefing. Our deep research feature performs web research with source citations. Contextual memory allows Anchorpoint to maintain user data across chats, storing it locally and injecting it into prompts to maintain continuity across chats. Users can also engage with Anchorpoint through custom voice output, utilizing text to speech responses with selectable voice options. Underpinning all of these features is a commitment to private personalization, with all user profile data stored and processed completely locally.

2.3 Constraints, Assumptions and Dependencies

Anchorpoint operates under a few key constraints, the most important being that all LLM inference must run on the user's local hardware, and that the system must be capable of running on consumer grade hardware with a minimum of 8 GB of RAM. We are assuming that users have a stable internet connection for features that require external API calls such as weather, news, and calendar sync, and that users have Ollama already installed or the system will guide them through installation. The target platform is desktop, supporting Windows, macOS, and Linux. In terms of dependencies, Anchorpoint relies on Ollama for local LLM inference, external APIs including Google Calendar, Tomorrow.io, NewsAPI, and Google Maps for commute data, a text to speech engine such as Piper for voice output, and ChromaDB as the vector database for contextual memory retrieval.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 Software Interfaces

The Ollama runtime provides all LLM inference for chat completions and model management. For the daily report, the system pulls from the Google Calendar API for the user's upcoming events, tomorrow.io for current weather and forecasts based on user location, NewsAPI for top headlines and topic-based articles, and the Google Routes API for estimated commute times based on the user's home and work locations. On the storage side, ChromaDB serves as the local vector database for storing and retrieving contextual memory embeddings that give the LLM long term recall. For voice output, a text to speech engine such as Piper converts LLM text responses into audio with configurable voice profiles.

3.2 Functional Requirements

3.2.1 Chat Interface

The system shall provide a real time chat interface where users can type messages, receive streamed LLM responses, and view conversation history within the current session.

3.2.2 Private Personalization Input

The system shall accept user input for private personalization and store it locally on the device for use in future LLM responses.

3.2.3 Daily Report Creation

The system shall create a daily report combining the user’s calendar events, local weather, top news headlines, and estimated commute time into a single briefing.

3.2.4 Deep Research

The system shall allow users to conduct deep research queries where the LLM searches the web, retrieves sources, and returns a summarized answer with citations.

3.2.5 Voice Output

The system shall generate audio responses using at least 3 selectable voice profiles with customizable tone and speaking speed.

3.2.6 Graceful Tool Failure

The system shall gracefully handle tool failures by returning a partial result with an explanation rather than crashing.

3.2.7 Contextual Memory

The system shall maintain a persistent contextual memory store that tracks user conversation across sessions with no data loss.

3.2.8 Model Selection and Management

The system shall allow users to select, download, and switch between available local LLM models through the settings interface.

3.2.9 Web Search

The system shall integrate a web search capability that retrieves and parses web content for use in both deep research and general chat queries.

3.2.10 Conversation History

The system shall persist conversation history locally and allow users to search, view, and delete past conversations.

3.3 Use Cases

3.3.1 Use Case #1

Use Case ID:	UC-1.0		
Use Case Name:	Daily Report		
Created By:	Dante Samarco		
Date Created:	3/2/2026		
Actors:	Frederick Godwin, 19 years old		
Description:	Daily report for Anchorpoint, checks current information		
Trigger:	Frederick wakes up to begin his day, and is looking to catch himself up with the morning news, weather, and any upcoming events in his calendar. Frederick needs to know this information quickly as he is running late for class.		
Preconditions:	1. Anchorpoint is installed onto device 2. LLM is already set up on device 3. LLM is familiar with Frederick's data		
Postconditions:	1. Frederick receives updates on news, weather, and upcoming events on his calendar in an appropriate manner		
Normal Flow:	1. Frederick wakes up 2. Frederick opens Anchorpoint on his laptop and asks for what the day is looking like 3. Anchorpoint delivers on current news, weather, and events coming up on his calendar in a timely manner 4. Anchorpoint asks if Frederick is satisfied with results		

	5. Frederick says yes and gets ready for his day
Alternative Flows:	4a. Anchorpoint does not deliver daily start in timely manner 1. Resumes for step 4 2. Frederick says no and only needs to know the weather 3. Anchorpoint takes feedback and learns to prioritize weather over anything else for Frederick's daily start 4. Use case resumes on step 5
Exceptions:	N/A
Includes:	N/A
Frequency of Use:	Daily, in the morning
Special Requirements:	Access to internet and device's internet browser app
Assumptions:	The daily report will provide accurate information based on search results
Notes and Issues:	Can users possibly personalize this daily report instead of a default?

3.3.2 Use Case #2

Use Case ID:	UC-2.0		
Use Case Name:	Recommend Model		
Created By:	Dante Samarco	**Last Updated By:**	
Date Created:	3/2/2026	**Last Revision Date:**	
Actors:	Mike Digiorno, 35 years old		
Description:	Getting a recommended LLM for their device.		
Trigger:	Mike is trying out Anchorpoint for the first time, after his large skepticism around LLMs. Mike is looking to get the best LLM for his device, an Apple MacBook Pro. Mike is currently at the step to where Anchorpoint reads his hardware and recommends what's best for his device.		
Preconditions:	1. Mike has a device with Anchorpoint installed onto his Apple smartphone 2. Mike has launched Anchorpoint on his device for the first time		
Postconditions:	1. Anchorpoint selects best LLM for device based on hardware specifications 2. LLM is installed onto device 3. Mike is ready to use Anchorpoint		
Normal Flow:	1. Mike installs Anchorpoint 2. Mike opens Anchorpoint for the first time 3. Anchorpoint prompts Mike to find best LLM or select his own 4. Anchorpoint validates files 5. Anchorpoint installed LLM and is ready to be used		
Alternative Flows:	3a. In step 3 of normal flow, Mike selects find best LLM 1. Anchorpoint reviews hardware specs 2. Anchorpoint searches for LLMs based on results 3. Anchorpoint prompts with best fit 4. Mike accepts 5. Use case resumes on step 4 3b. In step 3 of normal flow, Mike selects his own LLM 1. Mike uploads either URL or zip to Anchorpoint 2. Anchorpoint validates files 3. Anchorpoint asks if this is what Mike wants to use		

	4. Mike accepts 5. Use case resumes on step 4
Exceptions:	3a. In step 3b of the alternative flow, if Mike uploads incorrect URL or files 1. Files are unvalidated 2. Anchorpoint prompts to try again or to find best LLM for device 3. Mike selects try again 4. Use case resumes on step 1 of 3b in alternative flow
Includes:	N/A
Frequency of Use:	On demand
Special Requirements:	Access to internet, files already on device (if user is uploading)
Assumptions:	The search for best LLM for device grants optimal LLM for device
Notes and Issues:	N/A

3.4 Classes / Objects

We are not completely sold on using object oriented programming, as the ease of functional programming may outway the benefits of OOP. However, since we have not fully decided we have drafted out our classes / objects.

3.4.1 ChatSession

Represents a single conversation session between the user and the local LLM. Manages message history and provides the interface for adding and retrieving messages.

3.4.1.1 Attributes

- id : int # Unique session identifier
- title : string # Auto-generated or user-set session name
- createdAt : datetime # Timestamp of when the session was started
- messages : List<Message> # Ordered list of all messages in the session

3.4.1.2 Methods

- addMessage(role, content) : void
  # Appends a new Message object to the session's message list
- getHistory() : List<Message>
  # Returns the full ordered message list for prompt construction

3.4.2 Message

Represents a single message within a chat session. Stores the sender role, text content, and creation time.

3.4.2.1 Attributes

- id : int # Unique message identifier
- role : string # "user" or "assistant"
- content : string # The text body of the message
- timestamp : datetime # When the message was created

3.4.2.2 Methods

- None

3.4.3 UserProfile

Stores the local user's personal information and preferences used to personalize briefings, prompts, and memory injection.

3.4.3.1 Attributes

- id : int # Unique profile identifier
- name : string # User's preferred display name
- location : string # City/region used for weather API lookups
- commuteInfo : string # Home and work addresses for commute data

3.4.3.2 Methods

- update(field, value) : void
  # Updates a specified profile field and persists the change to local storage

3.4.4 MemoryManager

Manages Anchorpoint's cross-chat contextual memory system. Interfaces with ChromaDB to store, retrieve, and inject semantically relevant user facts into LLM prompts.

3.4.4.1 Attributes

- entries : List<MemoryEntry> # Currently loaded/cached memory entries

3.4.4.2 Methods

- injectMemoryIntoPrompt(prompt) : string
  # Embeds the prompt, queries ChromaDB for top-k relevant memories, and returns the augmented prompt string
- saveMemoryEntry(key, value) : void
  # Embeds a new memory fact and stores it in ChromaDB
- deleteMemoryEntry(id) : void
  # Removes a memory entry from ChromaDB by UUID

3.4.5 MemoryEntry

Represents a single persistent contextual fact about the user, stored as a vector document in ChromaDB.

3.4.5.1 Attributes

- id : string # UUID per entry
- document : string # The memory text, e.g. "User is a SWE student at Pitt"
- embedding : vector # Locally generated vector embedding of the document
- metadata.key : string # Optional category tag, e.g. "work", "schedule"
- metadata.source_conversation_id : string
  # UUID of the conversation where this memory was learned
- metadata.created_at : string # ISO 8601 timestamp

3.4.5.2 Methods

- None

3.4.6 LLMInterface

Provides the interface between Anchorpoint and the locally running Ollama backend. Handles prompt submission, streamed token responses, and model status checks.

3.4.6.1 Attributes

- modelName : string
  # Name of the currently loaded Ollama model (e.g. llama3, mistral)

3.4.6.2 Methods

- sendPrompt(prompt) : string
  # Sends a full prompt to Ollama and returns the complete response string
- streamResponse(prompt) : Stream
  # Sends a prompt and streams response tokens back to the UI as they are generated
- checkStatus() : bool
  # Returns true if Ollama is running and reachable, false otherwise

3.4.7 BriefingService

Orchestrates the generation of the user's personalized morning briefing by fetching external data, assembling a prompt, and invoking the local LLM.

3.4.7.1 Attributes

- None (stateless operates on UserProfile and external APIs)

3.4.7.2 Methods

- generateBriefing() : string
  # Orchestrates all data fetching and LLM prompt construction, returns the final briefing string
- fetchWeather() : string
  # Calls the external weather API with the user's location and returns a formatted weather summary
- fetchNews() : string
  # Calls the external news API and returns formatted top headlines
- fetchCalendarEvents() : List<string>
  # Retrieves locally stored calendar events and returns them as a list of strings

3.4.8 TTSService

Provides text to speech functionality for verbal output of LLM responses. Supports selectable voices and playback control.

3.4.8.1 Attributes

- selectedVoice : string # The currently active TTS voice identifier

3.4.8.2 Methods

- speak(text) : void
  # Converts the provided text to speech and plays it through the system audio output
- stop() : void
  # Halts any currently active speech playback
- listVoices() : List<string>
  # Returns the list of available TTS voice options

3.5 Non-Functional Requirements

3.5.1 Performance

- Time to first token shall be 3 seconds or less on systems with at least 8 GB of RAM.
- Context window capped at approximately 200,000 tokens.

3.5.2 Reliability

- All external API calls shall enforce a 10 second timeout.
- No unhandled exception shall reach the user during normal operation.

3.5.3 Availability

- Core LLM functionality including chat, memory, and personalization shall be available 99.9% of the time the machine is powered on with no external server dependency.
- Internet-dependent features shall remain operational within 5 seconds of network reconnection.

3.5.4 Security

- No personally identifiable information shall leave the device during any Anchorpoint operation.
- All locally stored user data shall be accessible only to the user's own system account.
- External API communication shall transmit only the minimum data required for the request and no user conversation content.

3.5.5 Maintainability

- Adding a new feature shall not require changes to more than one architectural layer.
- Each module shall maintain at least 50% unit test coverage.

3.5.6 Portability

- Anchorpoint shall support Windows, macOS, and Linux.
- Minimum hardware requirement is 8 GB of RAM.

3.6 Logical Database Requirements

Anchorpoint will use ChromaDB for its database, running entirely on the device locally. ChromaDB is preferable in how it allows for memorization of conversation history for relevant context. Data must remain local to the device.

The following collections help to define how the software will memorize context in order to carry over prior information to new conversations securely.

Collections:

memory_entries

Field	**Type**	**Notes**
id	string	UUID per entry
document	string	The memory text, e.g. "User is a SWE student at Pitt"
embedding	vector	Generated locally via embedding model
metadata.source_conversation_id	string	Which conversation this was learned from

metadata.created_at	string	ISO timestamp
metadata.key	string	Category tag, e.g. "work", "schedule"

conversation_history

Field	Type	Notes
id	string	UUID per message
document	string	Message content
embedding	vector	Generated locally
metadata.conversation_id	string	Session grouping
metadata.role	string	"user" or "assistant"
metadata.timestamp	string	ISO timestamp

briefing_cache

Field	Type	Notes
id	string	UUID
document	string	Full briefing text
embedding	vector	Generated locally
metadata.date	string	YYYY-MM-DD
metadata.generated_at	string	ISO timestamp

3.7 Other Requirements

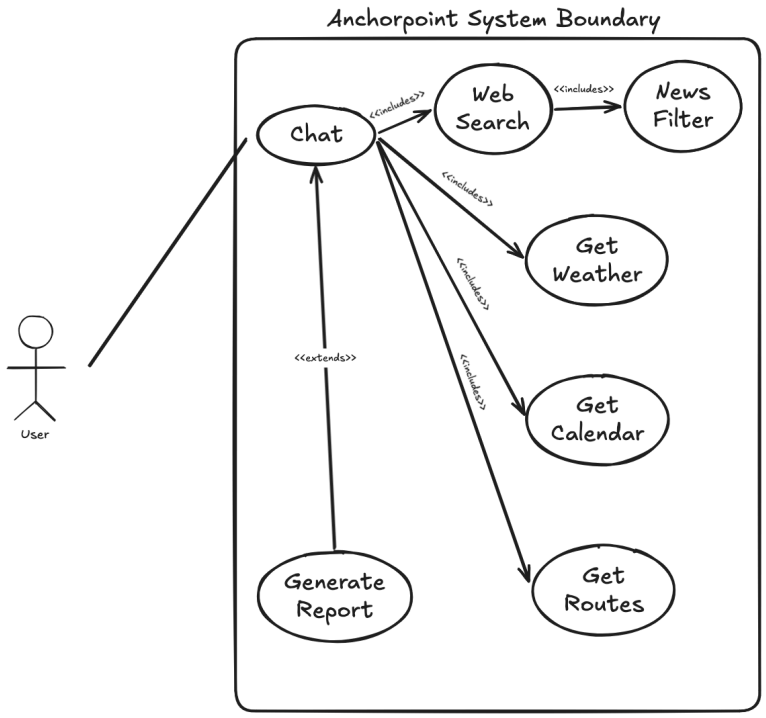
Catchall section for any additional requirements.

4. Analysis Models

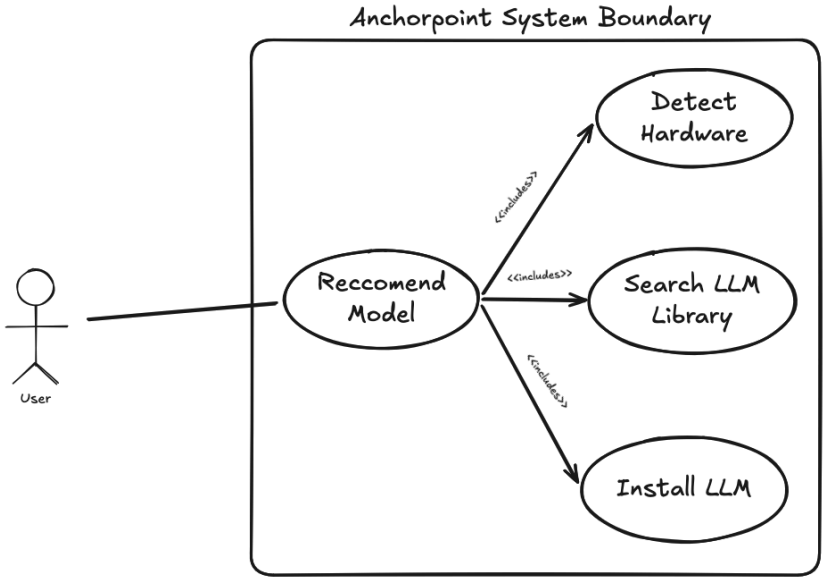
Only two diagrams shown as per rough draft specifications

4.1 Use Case Diagrams

4.1.1 Use Case #1 Diagram



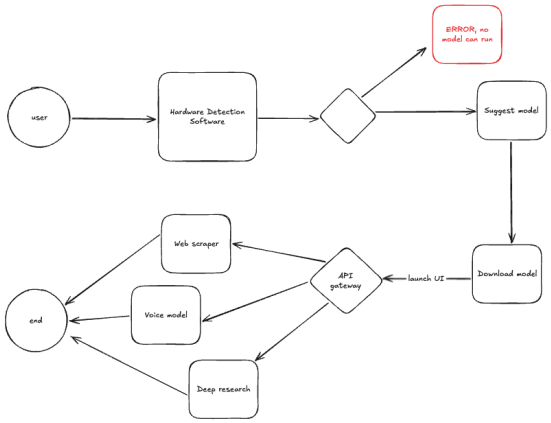
4.1.2 Use Case #2 Diagram



4.2 Class Diagrams

4.3 Sequence Diagrams

4.4 Activity Diagrams



4.5 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.